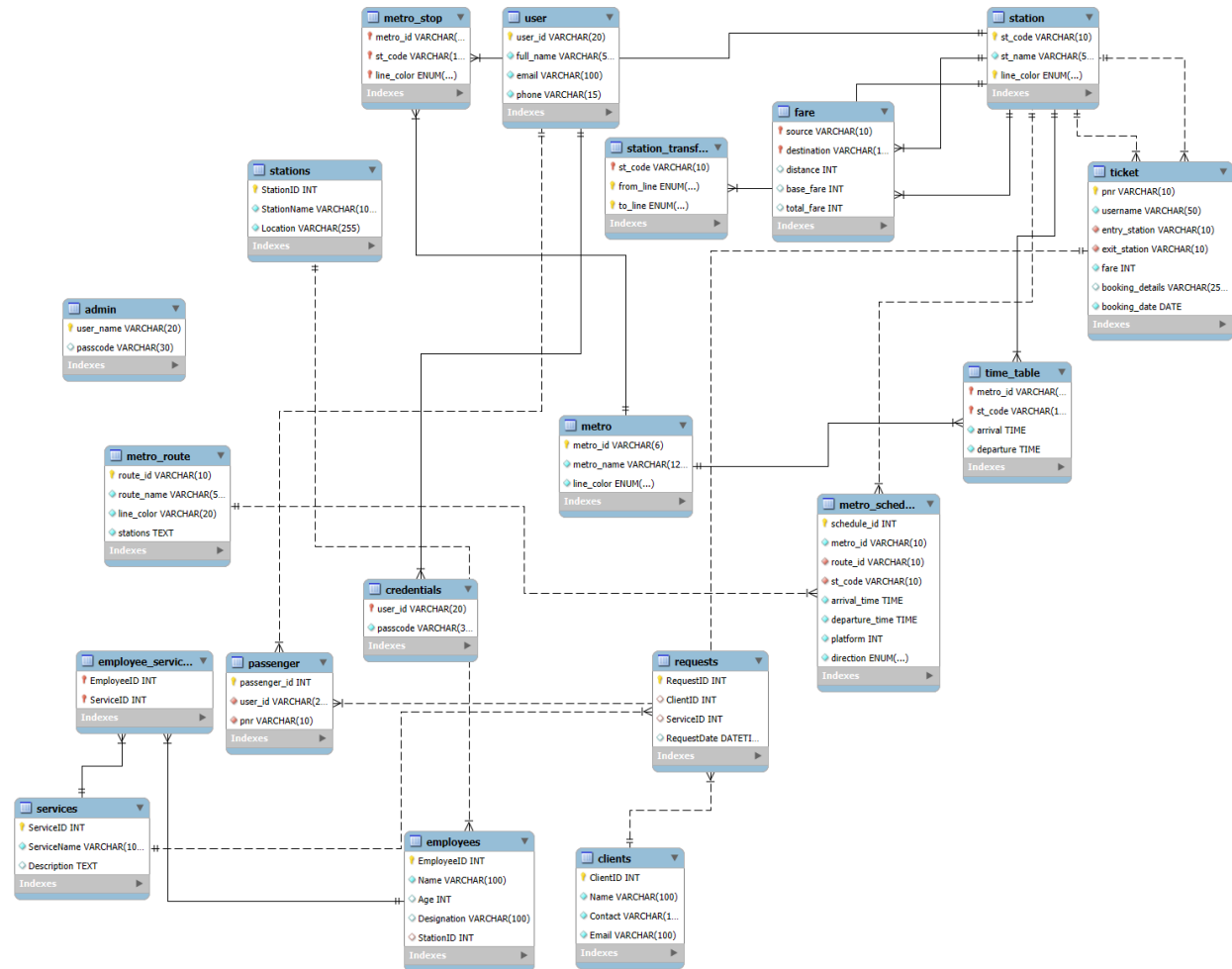


ER MODEL(UPDATED)



SQL Query

Query 1: Metro Schedule for a Specific Station and Time

Retrieves metro details at station 'S3' within the specified time range. Includes metro ID, name, route, platform, direction, arrival, and departure times.

RESULT

```

257 • SELECT m.metro_id, m.metro_name, ms.route_id, ms.arrival_time, ms.departure_time, ms.platform, ms.direction
258 FROM metro_schedule ms
259 JOIN metro m ON ms.metro_id = m.metro_id
260 WHERE ms.st_code = 'S3' -- Replace with the desired station code
261 AND '08:10:00' BETWEEN ms.arrival_time AND ms.departure_time; -- Replace with the desired time
262

```

	metro_id	metro_name	route_id	arrival_time	departure_time	platform	direction
▶	M14	Blue Metro 4	R1	08:10:00	08:12:00	1	Forward
	M12	Blue Metro 2	R1	08:10:00	08:12:00	2	Reverse

Query 2: Metro Travel Schedule Between Two Stations

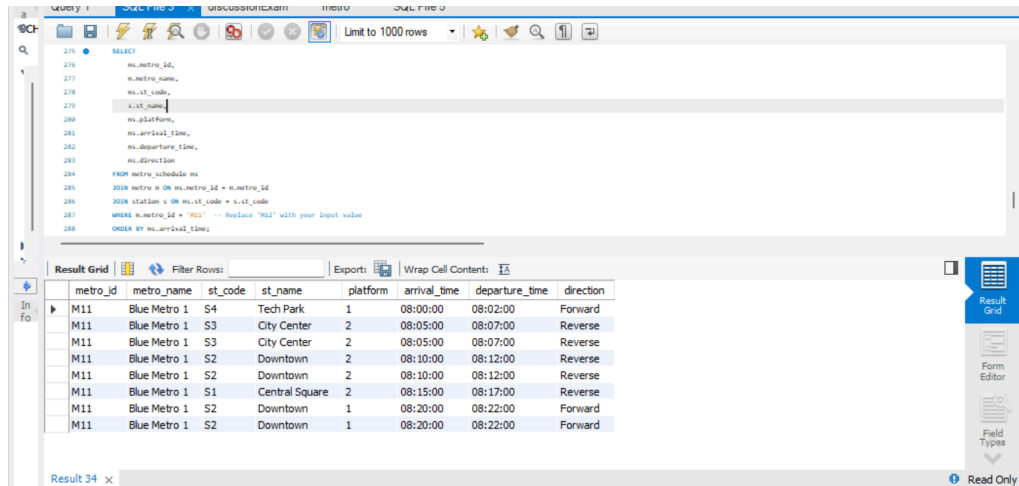
Finds metros traveling from 'S6' to 'S2', ensuring correct order of travel. Displays metro ID, name, start station/time, and end station/time, sorted by arrival time.

RESULT

metro_id	metro_name	start_station	start_time	end_station	end_time
M20	Pink Metro 2	S6	08:00:00	S2	08:15:00
M21	Pink Metro 3	S6	08:05:00	S2	08:20:00
M19	Pink Metro 1	S6	08:05:00	S2	08:10:00
M19	Pink Metro 1	S6	08:05:00	S2	08:20:00
M20	Pink Metro 2	S6	08:10:00	S2	08:15:00
M21	Pink Metro 3	S6	08:15:00	S2	08:20:00

Query 3: Metro Timetable for a Specific Metro

Displays the complete schedule of metro 'M11', listing station names, arrival/departure times, platform numbers, and direction. Ordered by arrival time.



```

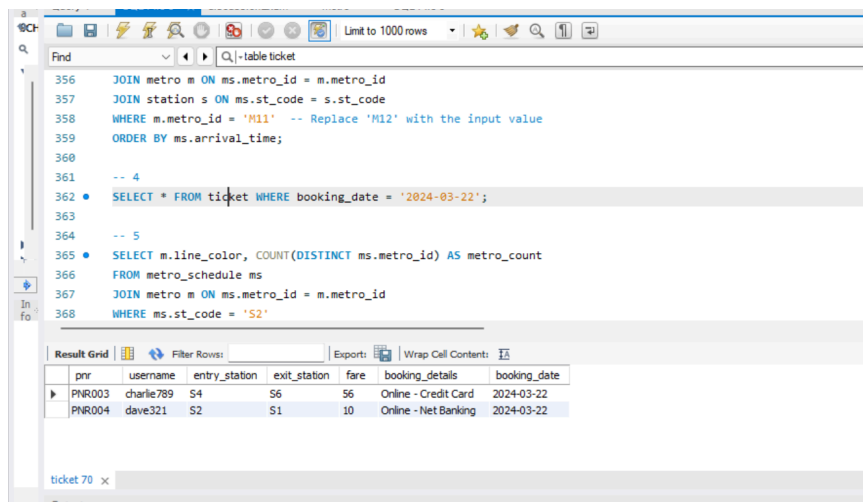
276 SELECT
277     m.metro_id,
278     m.metro_name,
279     m.st_code,
280     s.st_name,
281     m.platform,
282     m.arrival_time,
283     m.departure_time,
284     m.direction
285 FROM metro_schedule ms
286 JOIN metro m ON m.metro_id = ms.metro_id
287 JOIN station s ON ms.st_code = s.st_code
288 WHERE m.metro_id = 'M11' -- Replace 'M12' with your input value
289 ORDER BY ms.arrival_time

```

metro_id	metro_name	st_code	st_name	platform	arrival_time	departure_time	direction
M11	Blue Metro 1	S4	Tech Park	1	08:00:00	08:02:00	Forward
M11	Blue Metro 1	S3	City Center	2	08:05:00	08:07:00	Reverse
M11	Blue Metro 1	S3	City Center	2	08:05:00	08:07:00	Reverse
M11	Blue Metro 1	S2	Downtown	2	08:10:00	08:12:00	Reverse
M11	Blue Metro 1	S2	Downtown	2	08:10:00	08:12:00	Reverse
M11	Blue Metro 1	S1	Central Square	2	08:15:00	08:17:00	Reverse
M11	Blue Metro 1	S2	Downtown	1	08:20:00	08:22:00	Forward
M11	Blue Metro 1	S2	Downtown	1	08:20:00	08:22:00	Forward

Query 4: Tickets Booked on a Specific Date

Fetches all ticket details booked on '2024-03-22', including PNR, username, entry/exit stations, fare, booking details, and booking date.



```

356 JOIN metro m ON m.metro_id = m.metro_id
357 JOIN station s ON ms.st_code = s.st_code
358 WHERE m.metro_id = 'M11' -- Replace 'M12' with the input value
359 ORDER BY ms.arrival_time;
360
361 -- 4
362 SELECT * FROM ticket WHERE booking_date = '2024-03-22';
363
364 -- 5
365 SELECT m.line_color, COUNT(DISTINCT ms.metro_id) AS metro_count
366 FROM metro_schedule ms
367 JOIN metro m ON m.metro_id = ms.metro_id
368 WHERE ms.st_code = 'S2'

```

pnr	username	entry_station	exit_station	fare	booking_details	booking_date
PNR003	charlie789	S4	S6	56	Online - Credit Card	2024-03-22
PNR004	dave321	S2	S1	10	Online - Net Banking	2024-03-22

Query 5: Metro Count by Line Color at a Specific Station

Counts the number of unique metros stopping at station 'S2', grouped by metro line color. Helps analyze traffic density at the station.

Query 1 SQL File 3 discussionExam metro SQL File 5

Find Q | table ticket Done

```

360
361 -- 4
362 • SELECT * FROM ticket WHERE booking_date = '2024-03-22';
363
364 -- 5
365 • SELECT m.line_color, COUNT(DISTINCT ms.metro_id) AS metro_count
366 FROM metro_schedule ms
367 JOIN metro m ON ms.metro_id = m.metro_id
368 WHERE ms.st_code = 'S2'
369 GROUP BY m.line_color;
370
371 -- 6
372 • SELECT COUNT(*) AS passenger_count

```

Result Grid Filter Rows: Export: Wrap Cell Content: 15

line_color	metro_count
Blue	4
Pink	4

Result Grid

Query 6: Passenger Count on a Specific Journey

This query counts the total number of tickets where the entry station is 'S1' and the exit station is 'S6' on the date '2024-03-20'. It provides a straightforward measure of passenger volume for that particular journey on that day.

Find Q | table ticket Done

```

368 WHERE ms.st_code = 'S2'
369 GROUP BY m.line_color;
370
371 -- 6
372 • SELECT COUNT(*) AS passenger_count
373 FROM ticket
374 WHERE entry_station = 'S1'
375 AND exit_station = 'S6'
376 AND booking_date = '2024-03-20';
377
378 -- 7
379 • SELECT COUNT(DISTINCT p.passenger_id) AS total_passengers
380 FROM passenger p

```

Result Grid Filter Rows: Export: Wrap Cell Content: 15

passenger_count
1

Result Grid

Query 7: Distinct Passenger Count for Blue Line Journeys

This query counts the number of distinct passengers (via `passenger_id`) whose tickets involve at least one station on the Blue line (either entry or exit) on '2024-03-22'. It helps analyze Blue line usage by filtering tickets with Blue as either start or end.

```
378 -- 7
379 • SELECT COUNT(DISTINCT p.passenger_id) AS total_passengers
380 FROM passenger p
381 JOIN ticket t ON p.pnr = t.pnr
382 JOIN station s1 ON t.entry_station = s1.st_code
383 JOIN station s2 ON t.exit_station = s2.st_code
384 WHERE (s1.line_color = 'Blue' OR s2.line_color = 'Blue')
385 AND t.booking_date = '2024-03-22'; -- Replace with the required date
386
387 -- 8
388 -- most popular station , route between station
389 • SELECT entry_station, COUNT(*) AS entry_count
390 FROM ticket
```

total_passengers
2

Result 80 x

Query 8: Most Popular Entry Station

This query identifies the most popular entry station by counting the number of tickets grouped by the entry station and ordering the counts in descending order. It returns the station with the highest ticket entry count, indicating where most journeys begin.

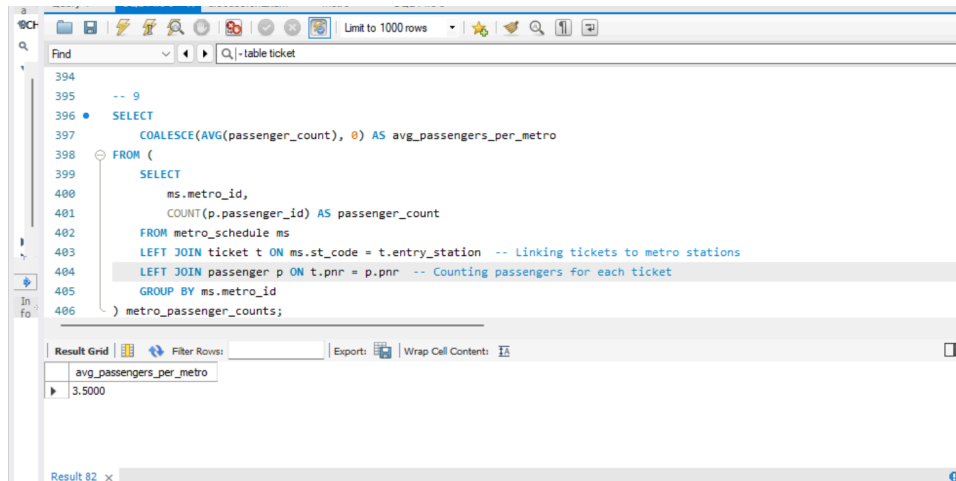
```
384 WHERE (s1.line_color = 'Blue' OR s2.line_color = 'Blue')
385 AND t.booking_date = '2024-03-22'; -- Replace with the required date
386
387 -- 8
388 -- most popular station , route between station
389 • SELECT entry_station, COUNT(*) AS entry_count
390 FROM ticket
391 GROUP BY entry_station
392 ORDER BY entry_count DESC
393 LIMIT 1;
394
395 -- 9
396 • SELECT
```

entry_station	entry_count
S2	2

Result 81 x

Query 9: Average Passengers per Metro

This query calculates the average number of passengers per metro by first counting passengers associated with each metro (using a LEFT JOIN on metro_schedule, ticket, and passenger tables) and then computing the overall average. It provides an insight into metro occupancy levels across the network.



```
394
395 -- 9
396 • SELECT
397     COALESCE(AVG(passenger_count), 0) AS avg_passengers_per_metro
398 FROM (
399     SELECT
400         ms.metro_id,
401         COUNT(p.passenger_id) AS passenger_count
402     FROM metro_schedule ms
403     LEFT JOIN ticket t ON ms.st_code = t.entry_station -- Linking tickets to metro stations
404     LEFT JOIN passenger p ON t.pnr = p.pnr -- Counting passengers for each ticket
405     GROUP BY ms.metro_id
406 ) metro_passenger_counts;
```

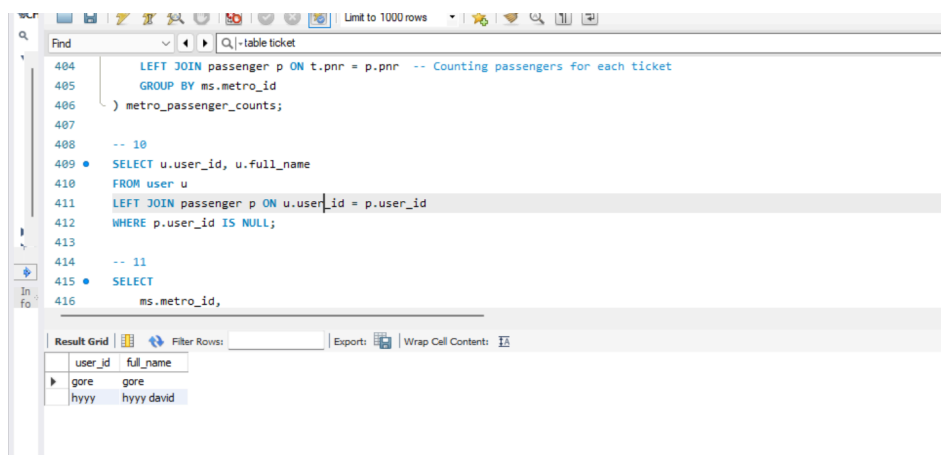
Result Grid

avg_passengers_per_metro
3.5000

Result 82 x

Query 10: Users Without Ticket Bookings

This query retrieves user IDs and full names for users who have not booked any tickets. It performs a LEFT JOIN between the **user** and **passenger** tables and filters out users with a corresponding booking, thus identifying inactive users in terms of ticket purchases.



```
404     LEFT JOIN passenger p ON t.pnr = p.pnr -- Counting passengers for each ticket
405     GROUP BY ms.metro_id
406 ) metro_passenger_counts;
407
408 -- 10
409 • SELECT u.user_id, u.full_name
410 FROM user u
411 LEFT JOIN passenger p ON u.user_id = p.user_id
412 WHERE p.user_id IS NULL;
413
414 -- 11
415 • SELECT
416     ms.metro_id,
```

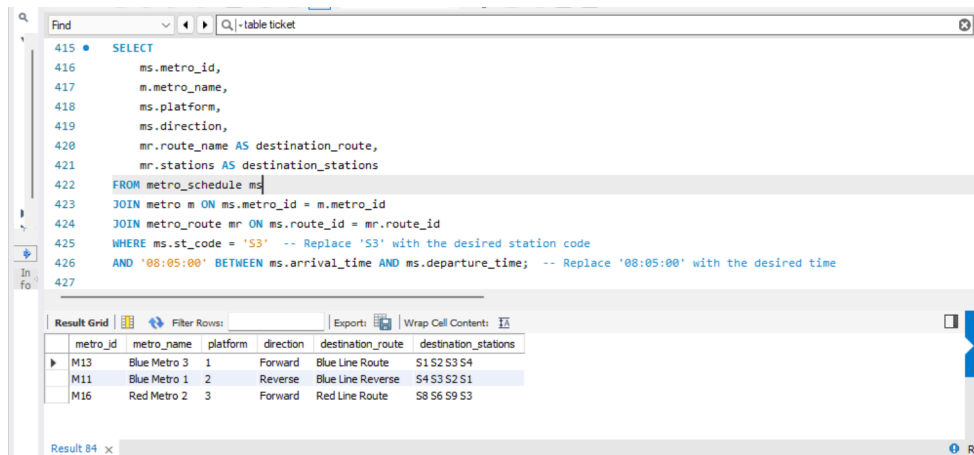
Result Grid

user_id	full_name
gore	gore
hyyy	hyyy david

Query 11: Metro Schedule Details for a Specific Station and Time

This query fetches schedule details (metro ID, metro name, platform, direction,

route name, and route stations) for metros that are scheduled to stop at station 'S3' at a specified time ('08:05:00'). It joins the metro_schedule, metro, and metro_route tables to provide comprehensive route and timing details.



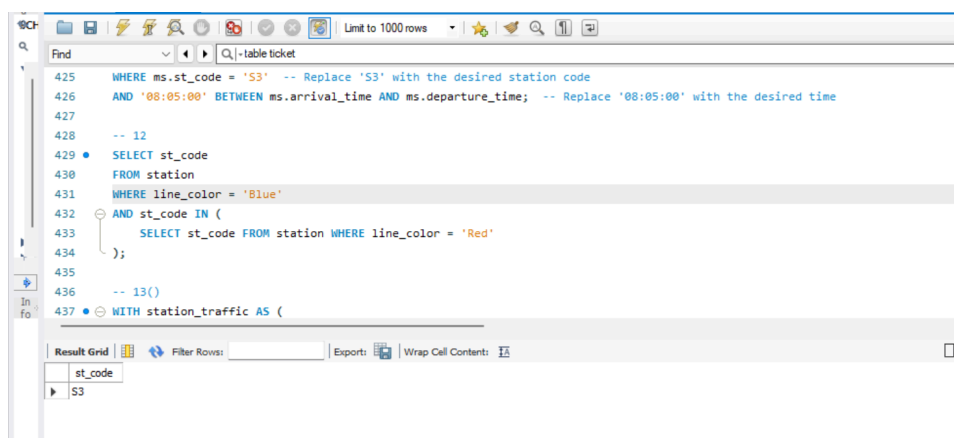
```
415 SELECT
416     ms.metro_id,
417     m.metro_name,
418     ms.platform,
419     ms.direction,
420     mr.route_name AS destination_route,
421     mr.stations AS destination_stations
422 FROM metro_schedule ms
423 JOIN metro m ON ms.metro_id = m.metro_id
424 JOIN metro_route mr ON ms.route_id = mr.route_id
425 WHERE ms.st_code = 'S3' -- Replace 'S3' with the desired station code
426 AND '08:05:00' BETWEEN ms.arrival_time AND ms.departure_time; -- Replace '08:05:00' with the desired time
427
```

metro_id	metro_name	platform	direction	destination_route	destination_stations
M13	Blue Metro 3	1	Forward	Blue Line Route	S1 S2 S3 S4
M11	Blue Metro 1	2	Reverse	Blue Line Reverse	S4 S3 S2 S1
M16	Red Metro 2	3	Forward	Red Line Route	S8 S6 S9 S3

Result 84 x

Query 12: Stations on Both Blue and Red Lines

This query selects station codes that appear on both the Blue and Red lines. It uses an IN clause to filter stations listed as 'Blue' that also exist in the set of stations listed as 'Red', identifying stations serving multiple lines.



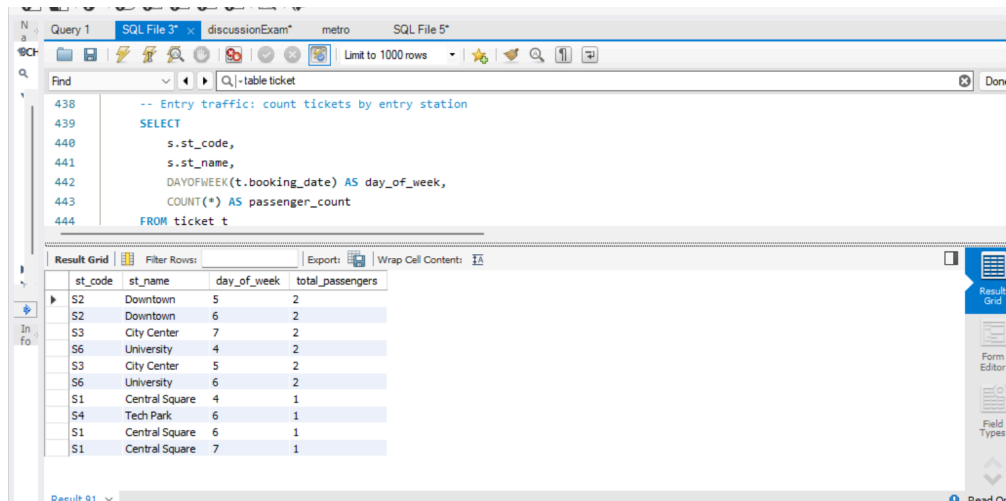
```
425 WHERE ms.st_code = 'S3' -- Replace 'S3' with the desired station code
426 AND '08:05:00' BETWEEN ms.arrival_time AND ms.departure_time; -- Replace '08:05:00' with the desired time
427
428 -- 12
429 SELECT st_code
430 FROM station
431 WHERE line_color = 'Blue'
432 AND st_code IN (
433     SELECT st_code FROM station WHERE line_color = 'Red'
434 );
435
436 -- 13()
437 WITH station_traffic AS (
```

st_code
S3

Query 13: Traffic Analysis by Station, Day, and Hour

This query aggregates traffic data by combining entry and exit counts per station,

day of week. Using a UNION ALL to merge counts from both entry and exit perspectives, it then groups and sums these counts, ordering the results by total passenger count in descending order to rank stations by traffic volume.



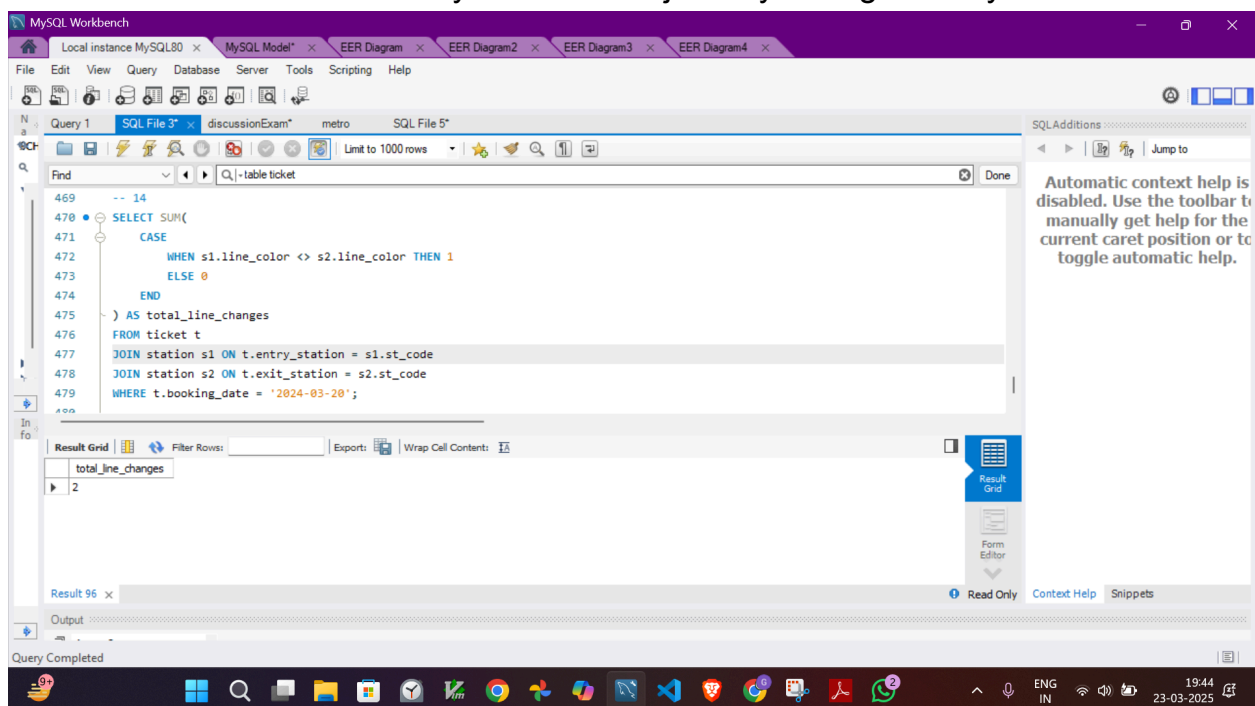
The screenshot shows a SQL IDE window with a query editor and a result grid. The query is as follows:

```
-- Entry traffic: count tickets by entry station
SELECT
    s.st_code,
    s.st_name,
    DAYOFWEEK(t.booking_date) AS day_of_week,
    COUNT(*) AS passenger_count
FROM ticket t
```

The result grid displays the following data:

st_code	st_name	day_of_week	total_passengers
S2	Downtown	5	2
S2	Downtown	6	2
S3	City Center	7	2
S6	University	4	2
S3	City Center	5	2
S6	University	6	2
S1	Central Square	4	1
S4	Tech Park	6	1
S1	Central Square	6	1
S1	Central Square	7	1

Query 14: This query returns the total number of passengers who have changed the metro lines from which they started their journey on a given day.



The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query is as follows:

```
-- 14
SELECT SUM(
    CASE
        WHEN s1.line_color <> s2.line_color THEN 1
        ELSE 0
    END
) AS total_line_changes
FROM ticket t
JOIN station s1 ON t.entry_station = s1.st_code
JOIN station s2 ON t.exit_station = s2.st_code
WHERE t.booking_date = '2024-03-20';
```

The result grid displays the following data:

total_line_changes
2